



Plataforma Educativa con Inteligencia Artificial



eduiaugb.streamlit.app | v1.0.0 

Equipo de Desarrollo:

Jonathan Roberto Acosta López • Rene Alexander Araujo Soto • Mario Alexander Hernández Quevedo

Universidad Gerardo Barrios — Módulo 4

Problema, Usuario y Valor

El Problema

Los docentes pierden horas creando evaluaciones y dando retroalimentación individual. Los estudiantes reciben contenido estático sin adaptación a su nivel.

Usuarios Objetivo



Docente: Necesita automatizar evaluaciones y feedback.



Estudiante: Necesita retroalimentación inmediata y práctica adaptada.



Administrador: Necesita gestionar la plataforma eficientemente.

Valor que entrega EduIA

- ✓ Genera exámenes desde el material del curso
- ✓ Evalúa código con feedback inmediato
- ✓ Chat educativo con el material como contexto (RAG simplificado)



Evolución de la Arquitectura

Sesión 2-4: API + Docker

Desacoplamiento: Streamlit UI →
FastAPI → Google Gemini.
Integración de contenedores
Docker.

Mejora Sesión 6

Migración completada:
`google-generativeai` → `google.generativeai`
en `utils_ai_core.py`. Sin
FutureWarnings en producción.

Sesión 1: Diagnóstico

App **Streamlit** monolítica acoplada
con **SQLite** local. Problemas de
escalabilidad y mantenibilidad
iniciales.

Sesión 5-6: Producción

Streamlit Cloud → FastAPI + Rate
Limiting + Cache → Google Gemini
→ **Supabase PostgreSQL**.

Flujo Inteligente y Contrato API

Endpoints Principales de Inteligencia Artificial

Endpoint	Entrada	Salida
POST /api/evaluate	código + lenguaje	score 0-10 + feedback
POST /api/courses/generate	lenguaje + nivel	estructura de temas
POST /api/chat/ask	contexto + pregunta	respuesta Markdown

Códigos de Validación

-  **422 Unprocessable Entity:** Payload inválido (lenguaje no soportado, código vacío).
-  **503 Service Unavailable:** Gemini API no disponible.
-  **429 Too Many Requests:** Rate limit excedido (60 req/min por IP).

Técnica IA: Prompting con Contexto (RAG simplificado)

El material completo del módulo se incluye dinámicamente en el prompt del sistema. **Gemini** es instruido estrictamente para responder *solo* basándose en el contenido provisto, evitando alucinaciones fuera del temario.

Pruebas Automatizadas y CI/CD

111

Pruebas Automatizadas (Todas en ● Verde)

Archivo Tests	Cantidad
test_api_unit.py	29
test_analizador_sintaxis.py	26
test_analizador_logica.py	18
test_engagement_managers.py	38

Pipeline GitHub Actions

-  **Triggers:** Se activa en cada push a main y migracion-base-de-datos.
-  **3 Jobs paralelos:** Pruebas Unitarias + BD SQLite + BD PostgreSQL.
-  **Seguridad:** Variables de entorno inyectadas de forma segura (GEMINI_API_KEY nunca expuesta en código).



Despliegue e Infraestructura

Stack de Producción (Costo Operativo: \$0)

Componente	Plataforma	Plan
 App Streamlit	Streamlit Cloud	Gratuito
 Base de Datos	Supabase PostgreSQL	Gratuito
 Repositorio	GitHub	Gratuito
 IA Engine	Google Gemini	Free Tier

Artefactos y Docker

```
# Construcción de la imagen local          docker build -t eduia-api:1.0 .
# Ejecución del contenedor API            docker run -p 8000:8000 \
-e GEMINI_API_KEY=your_key_here \ eduia-api:1.0
```

URL Pública: <https://eduiaiugb.streamlit.app/>

Rama Activa: **migracion-base-de-datos**

Release: v1.0.0 — commit 4ec811d

Observabilidad y Rendimiento

Instrumentación del Sistema

-  X-Request-ID único generado y trazado en cada request.
-  **Logs Estructurados:** request_id | method | path | status | duration_ms
-  **Rate Limiting:** 60 req/min (Rutas IA) / 300 req/min (Sistema General).
-  **AI**Cache: TTL de 24h para reducir llamadas repetidas y costos de API.
-  **APIKeyPool:** Sistema de rotación automática para distribuir carga en hasta 5 keys.

Métricas de Carga (Locust Base)

Escenario	Usuarios	p50 (Mediana)	p95	Error Rate
Prueba mínima	2	2 ms	4 ms	0%
Prueba de estrés	200	3 ms	8 ms	0%

[!] Cuello de botella identificado: Latencia de red hacia Google Gemini API (2 a 15 segundos por llamada de generación).

Plan de mitigación (Implementado):

Caché de respuestas → Pool de Keys → Migración a PostgreSQL (Múltiples instancias).

Seguridad, Release y Rollback

Controles Implementados

-  Secretos resguardados en Streamlit Cloud Secrets (Cero exposición en repo).
-  Validación paramétrica estricta vía Pydantic (Retorna 422 seguro).
-  Ocultación absoluta de stacktraces al cliente en errores 500.
-  Security headers HTTP (X-Frame-Options, HSTS, etc.).
-  Logging a archivo rotativo con trazabilidad por request_id.
-  Agente supervisor con alertas automáticas a Discord.

Gestión de Release (v1.0.0)

Riesgo Prioritario (Demo)

Cuota de Gemini agotada durante evaluación en vivo.

Control: Pool de 5 keys activo + fallback con mensaje UI amigable.

-  **Tag Release:** v1.0.0 en GitHub.
-  **Commit Demostrado:** 4ec811d en migracion-base-de-datos.
-  **Protocolo Rollback:** git revert al commit anterior → Streamlit Cloud detecta el cambio y redespiega automáticamente en ~2 minutos sin intervención manual.

Limitaciones Actuales y Lecciones Aprendidas

Limitaciones Conocidas

- **Sin autenticación JWT:** La API es pública actualmente sin control de acceso granular por token.
- El entorno gratuito de Streamlit Cloud suspende la aplicación tras 7 días sin tráfico web continuo.
- Transición técnica de base de datos de SQLite (local) a PostgreSQL (producción) parcialmente en curso (rama migracion-base-de-datos).

Lo Que Aprendimos

- ✓ Integrar LLMs impredecibles mediante prompting contextual rígido en una aplicación real.
- ✓ Implementar un flujo de integración y despliegue continuo (CI/CD) profesional con GitHub Actions.
- ✓ La enorme diferencia arquitectónica, operativa y de seguridad entre "funciona en mi máquina local" y un producto desplegado públicamente en la nube.

Demostración **En Vivo**

Plataforma desplegada y operativa en Streamlit Cloud.

<https://eduiaiugb.streamlit.app/>